ContractKV: Certified Prompt Layout Compilation and Transactional Prefix Reuse for Multi-Tenant LLM Services

Bin Zhou and Xi Cao, *Member, IEEE*

***Abstract*—Prefix caching accelerates LLM serving only when reusable tokens form a contiguous physical prefix. Repeated agent, retrieval, and code services violate this assumption: across five workflows, 85.0% of prompt tokens are stable, while native automatic prefix caching recovers only 2.4%. We present ContractKV, which converts this latent stability into verified physical reuse. A dependency-aware compiler constructs a bounded ladder of progressively more aggressive stable-prefix layouts; prospective qualification freezes the admissible ladder before held-out evaluation; strict teacher-forced attestation and a versioned scope key authorize reuse; and Prefix-Selective Transactional Promotion publishes only exact-LCP, block-aligned stable state. A compact in-engine attestation channel reduces verification egress by 2,148×, while service-template certificates amortize deployment attestation across authorized tenants. On vLLM 0.10.2, four model configurations from three families reduce all-workflow net mean TTFT by 29.41–44.69% after charging unhidden verification and raw fallback, with zero policy-attributable quality violations. On 30 identical frozen RULER sessions, three model families achieve 85.97–87.41% reduction. At 25 tenants per template, ContractKV reduces Qwen3-8B mean TTFT by 92.62% (95% CI: 91.68–93.31%) and improves 500-ms SLO goodput by 4.81×; at 80 tenants it reduces peak used KV blocks by 72% while sustaining approximately 11 compliant requests/s. Across 1,350 paired deployment-object outputs, task-score delta is zero. Failed verification or scope checks publish no shared state and fall back to raw order.**

***Index Terms*—KV cache, LLM inference serving, prefix caching, service-level objectives, multi-tenant consolidation, behavioral certification, transactional reuse.**

I. Introduction

Large language model (LLM) inference increasingly serves repeated agent, retrieval, and tool-augmented workflows under latency service-level objectives (SLOs) [1], [2]. We target deployments in which authorized requests repeatedly consume a stable service template and its prefix remains resident often enough to amortize certification. Because prefill work grows with prompt length, modern engines widely deploy prefix caching: they physically reuse the key-value (KV) states of token-identical prefixes [1], [2], lowering time-to-first-token (TTFT) without changing model semantics.

B. Zhou is with the College of Information Science and Technology, Beijing University of Chemical Technology, Beijing 102218, China (e-mail: 2025200800@buct.edu.cn).

X. Cao is with the College of Information Science and Technology, Beijing University of Chemical Technology, Beijing 102218, China (e-mail: caoxi@buct.edu.cn).

Corresponding author: Xi Cao.

Prefix caching nevertheless has a structural blind spot in increasingly common structured services. Agent pipelines [17], retrieval-augmented generation (RAG), and multi-turn tool use [18] compose prompts from heterogeneous segments—system instructions, retrieved documents, repository context, observations, and conversation state—whose application order need not match their reuse potential. One changed observation between two stable segments terminates the physical prefix even when all downstream stable content is byte-identical. Across five representative service families, stable content accounts for 78–91% of prompt tokens, whereas native caching recovers under 3% in raw arrival order. The resulting gap between an *application-declared byte-stable working set* and *physical prefix continuity* creates redundant prefill work that compounds with context length and tenant fan-out.

Moving stable segments ahead of volatile ones can expose this reuse, but naive reordering is unsafe. Segment order encodes dependencies such as chronological observations and code-context precedence; even a dependency-valid layout may alter position-sensitive generation. Once an unverified state enters a shared cache, one request's behavioral drift can affect every later consumer. Existing systems provide useful pieces—modular prompt reuse [7], [8], prefix indexing [2], [9], and KV compression or eviction [10]–[12]—but do not jointly enforce dependency validity, model-backed behavioral evidence, certificate scope, and atomic publication of compiled prefix state.

We present ContractKV, which resolves this tension through three coordinated mechanisms inside the serving engine's prefix-cache substrate. (1) A *Cache-Aware Layout Compiler* (CALC) represents each request as a typed segment dependency graph and constructs a bounded ladder of dependency-valid layouts with increasing stable-prefix promotion. (2) *Scoped Behavioral Certification* prospectively qualifies model–workflow–layout routes, then uses strict teacher-forced token-path agreement and a versioned certificate object to admit only in-scope candidates. (3) *Prefix-Selective Transactional Promotion* (PSTP) keeps candidate KV state ephemeral and publishes only the verified, exact-LCP, block-aligned stable prefix at a scheduler-thread linearization point. Any failure publishes no candidate compiled state; subsequent raw-order execution retains native APC behavior. A service-template certificate amortizes one deployment attestation across authorized tenants while private observations and

suffix state remain isolated.

We evaluate CONTRACTKV on vLLM 0.10.2 using Qwen3-8B, Qwen3-14B (TP=2), Mistral-7B-Instruct-v0.3, and Llama-3.1-8B under controlled traces and open-loop multi-tenant workloads. Across all workflows and three fresh-process repetitions, net mean TTFT falls by 29.41–44.69% after charging unhidden verification and all raw fallback, with zero scope-invalid admissions and zero policy-attributable quality violations. A common frozen RULER scope yields 86.67% macro reduction (CI [86.44, 86.90]). At 25 tenants per template, Qwen3-8B achieves 92.62% mean TTFT reduction and 4.81× SLO-goodput improvement; at 80 tenants, physical sharing reduces peak used KV blocks by 72% with zero unauthorized cross-tenant KV reuse.

This paper makes the following contributions:

- We formulate the *structured-prompt prefix-maximization problem* for LLM serving and introduce the segment dependency graph with typed stability annotations, establishing a formal basis for dependency-constrained prefix-oriented layout compilation (Section III).
- We design CONTRACTKV, combining cache-aware layout compilation, explicitly scoped behavioral certificates, and prefix-selective transactional promotion. The fail-closed protocol prevents failed or out-of-scope candidates from changing shared cache visibility (Section IV).
- We implement the pipeline in the vLLM V1 scheduler path. A compact source-level attestation channel reduces verification egress 2,148×, and a service-template certificate changes the verification object from a session to an authorized deployment template (Section V).
- We evaluate four model configurations from three families across single-tenant latency, open-loop SLO goodput, 80-tenant consolidation, physical KV-block deduplication, locality boundaries, and common-scope cross-model convergence. All admitted layouts satisfy the frozen quality and isolation gates (Sections VI–VII).

II. BACKGROUND AND MOTIVATION

A. Prefix Caching in LLM Serving

Modern LLM serving engines decompose transformer [20] inference into a **prefill** phase—which computes key-value (KV) states for prompt tokens via optimized attention kernels [21]—and an autoregressive **decode** phase. Schedulers [3], [6] interleave both phases across concurrent requests. For long contexts, many deployments enable *prefix caching*: when a token sequence shares a prefix with resident state, the engine reuses those KV blocks and computes only the suffix [1], [2].

Formally, let $\mathbf{t} = (t_1, \ldots, t_n)$ be a request's token sequence and $\mathcal{R}$ the set of resident KV states in the cache. The engine identifies the longest prefix match

$$k^* = \max\{k : (t_1, \ldots, t_k) \in \mathcal{R}\}, \tag{1}$$

reuses k^* blocks, and computes only the remaining $n - k^*$ tokens. Under *automatic prefix caching* (APC) in vLLM [1] or *RadixAttention* in SGLang [2], this matching is performed at block granularity B (typically 16 tokens), yielding physical reuse of $\lfloor k^*/B \rfloor$ blocks.

This mechanism is highly effective for *homogeneous* workloads—batch inference over identical system prompts, or multi-turn conversations where each turn appends to a growing prefix. In these settings, prefix reuse rates exceed 90% and TTFT reductions are substantial.

TABLE I
STRUCTURAL BLIND SPOT: STABLE CONTENT VASTLY EXCEEDS PHYSICAL PREFIX RECOVERY ACROSS FIVE SERVICE FAMILIES (QWEN3-8B, 10-TURN SESSIONS).

Service Family	Stable Ratio ρ_s	Native Recovery ρ_p
RULER (NIAH)	91.2%	2.8%
HotpotQA	82.4%	2.1%
NarrativeQA	78.6%	1.9%
Qasper	84.1%	2.4%
RepoBench	88.7%	2.6%
Mean	**85.0%**	**2.4%**

B. The Structural Blind Spot

Production LLM *services*, however, are increasingly *structured*: a single request composes multiple semantic segments—system instructions, retrieved documents, repository context, tool observations, and conversation state—whose physical ordering in the prompt is dictated by application logic rather than reuse potential. We term the resulting inefficiency the **structural blind spot**: the cache can only exploit *physical prefix continuity*, but the *semantic working set* that is actually reusable is distributed across non-contiguous positions.

Motivating measurement. We instrumented five representative service families (RULER, HotpotQA, NarrativeQA, Qasper, RepoBench) with 25 sessions each, 10 turns per session. For each request we compute:

- *Stable-token ratio* ρ_s: fraction of prompt tokens belonging to segments that remain byte-identical across turns within the same session.
- *Native prefix recovery* ρ_p: fraction of stable tokens actually recovered by the engine's native APC under raw arrival order.

Table I reports the results. The gap is stark: stable content constitutes 78–91% of prompt tokens, yet native prefix caching recovers fewer than 3% of this available reuse. The root cause is architectural: a single mutable segment (e.g., a tool observation) inserted between two large stable segments *fractures* the physical prefix, rendering all downstream stable content invisible to the cache despite being computationally identical.

This gap appears whenever an application interleaves byte-stable context with per-request observations and revisits the same authorized template. The degree of benefit is workload dependent: reuse must survive declared dependencies, behavioral admission, and cache residency. When those conditions hold, each unrecovered stable token incurs redundant prefill work, occupies scheduler slots, and consumes TTFT budget.

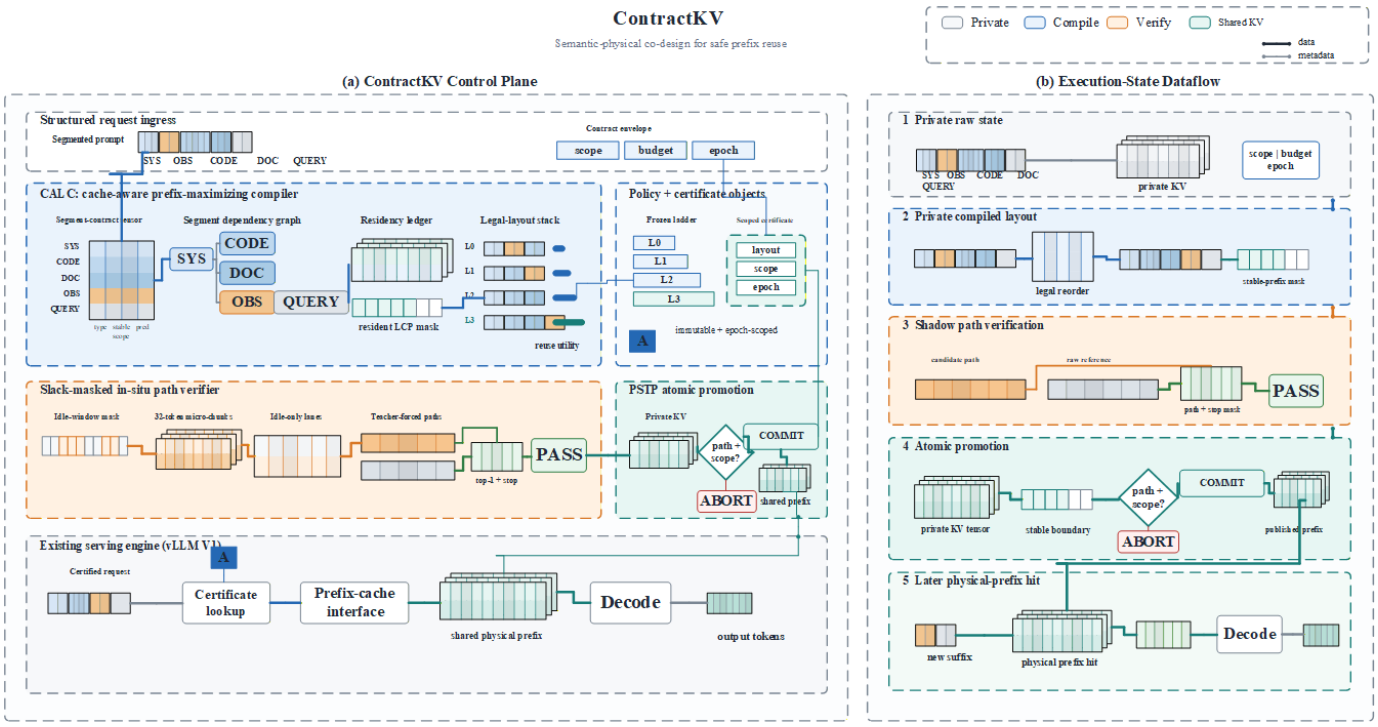


Fig. 1. ContractKV control-plane architecture and execution-state dataflow. (a) CALC combines segment contracts, the SDG, resident-prefix state, and qualification metadata; the verifier and PSTP publish only behaviorally certified compiled blocks into the vLLM cache domain. Solid arrows carry requests or KV state, whereas dashed arrows carry policy and certificate metadata. (b) A selected request evolves from private raw tokens to a private compiled layout, shadow verification, atomic promotion, and a later physical-prefix hit. A failed promotion discards only candidate compiled state; the raw-order path may populate native APC state as in unmodified vLLM.

C. Why Naive Reordering Is Inadmissible

A natural response is to *reorder* stable segments ahead of volatile ones, exposing a longer physical prefix. While this recovers reuse in principle, it introduces two correctness hazards that make unconstrained reordering unacceptable in a shared serving system:

Hazard 1: Dependency violation. Segment order encodes semantic dependencies. A tool observation may depend on a preceding system instruction; a code completion requires repository context before the query; a chronological agent history must preserve causal ordering. Arbitrary permutation violates these constraints, producing prompts that are cache-friendly but *semantically ill-formed*.

Hazard 2: Behavioral drift. Even when all dependencies are satisfied, a reordered prompt may shift the model's output distribution. Transformer attention is position-sensitive: moving a segment from position i to position j changes its attention pattern and can alter generation. Publishing an unverified reordered prefix into a shared cache makes this behavioral consequence *visible to all subsequent requests*—a silent, system-wide correctness risk with no rollback mechanism.

The fundamental tension. Exploiting structural reuse requires layout control; safe layout control requires verification; and reusable verification requires explicit scope and transactional publication. To our knowledge, no prior system jointly provides these properties. Figure 1 shows how ContractKV connects the compiler, certifier, and cache publication path.

III. Service Model and Problem Formulation

We now formalize the structured-prompt prefix-maximization problem that ContractKV solves. The formulation proceeds in three layers: a segment model with typed stability annotations, a dependency-constrained layout optimization objective, and a multi-tenant service-template abstraction.

A. Structured Request and Segment Model

Definition 1 (Structured Request): A **structured request** r is an ordered tuple of *semantic segments*

$$r = (s_1, s_2, \ldots, s_m), \quad s_i = (id_i,\ \tau_i,\ \sigma_i,\ \mathbf{c}_i), \tag{2}$$

where id_i is a unique segment identifier, $\tau_i \in \mathcal{T} = \{\textsc{Stable}, \textsc{Mutable}, \textsc{Query}\}$ is the **stability type**, σ_i is a content hash, and $\mathbf{c}_i$ is the token sequence of segment s_i.

The stability type captures the segment's reuse potential within a service session:

- STABLE: content is byte-identical across all turns in the session (system prompts, retrieved documents, repository context). These segments constitute the *reusable working set*.
- MUTABLE: content changes across turns but follows a predictable lifecycle (tool observations, intermediate results). These segments *fracture* prefix continuity.
- QUERY: the per-turn user query or action. Always unique; never reusable.

Let $L(r) = \sum_{i=1}^{m} |\mathbf{c}_i|$ denote the total prompt length and $L_s(r) = \sum_{i:\tau_i=\text{STABLE}} |\mathbf{c}_i|$ the stable-token length. The *structural reuse potential* is $\rho_s(r) = L_s(r)/L(r)$.

B. Segment Dependency Graph

Definition 2 (Segment Dependency Graph): The **segment dependency graph** (SDG) of a request r is a directed acyclic graph $G_r = (V, E, \lambda)$ where:

- $V = \{s_1, \ldots, s_m\}$ is the set of segment vertices;
- $E \subseteq V \times V$ is the set of *typed dependency edges*—$(s_i, s_j) \in E$ indicates that segment s_i must precede s_j in any valid layout;
- $\lambda : E \to \{\text{DATA}, \text{CAUSAL}, \text{POSITIONAL}\}$ labels each edge with its dependency type.

Edge types carry distinct semantics: DATA edges encode information flow (a tool result depends on its input); CAUSAL edges encode temporal ordering (turn k depends on turn $k-1$); POSITIONAL edges encode model-specific sensitivity (instruction must precede query). The SDG is constructed by the layout compiler from application-declared segment metadata and inferred ordering constraints.

Definition 3 (Layout): A **layout** π of request r is a permutation of V such that π is a *topological ordering* of G_r:

$$\forall (s_i, s_j) \in E: \quad \text{pos}_\pi(s_i) < \text{pos}_\pi(s_j). \tag{3}$$

The set of all valid layouts is $\Pi(G_r) = \{\pi : \pi \text{ is a topological order of } G_r\}$.

C. Layout Validity and Prefix Maximization

Given a layout π, the *compiled prompt* is the concatenation $\mathbf{c}_\pi = \mathbf{c}_{\pi(1)} \| \mathbf{c}_{\pi(2)} \| \cdots \| \mathbf{c}_{\pi(m)}$. Let $q(\pi)$ be the number of leading stable segments and $P(\pi)$ their token length:

$$\begin{aligned} q(\pi) &= \max\{q : \forall j \le q,\ \tau_{\pi(j)} = \text{STABLE}\}, \\ P(\pi) &= \sum_{j=1}^{q(\pi)} |\mathbf{c}_{\pi(j)}|. \end{aligned} \tag{4}$$

The physical reuse at block granularity B is $\lfloor P(\pi)/B \rfloor$ blocks.

Problem 1 (Dependency-Constrained Prefix Maximization): Given a structured request r with SDG G_r, resident state $\mathcal{R}$, and a finite compiler ladder $\Pi_L(G_r) = \{\pi_0, \ldots, \pi_K\} \subseteq \Pi(G_r)$, select the candidate that maximizes expected reuse among candidates admitted by the frozen verifier and scope policy:

$$\pi^* = \arg \max_{\pi \in \Pi_L(G_r)} \quad \mathbb{E}[\textit{reuse}_B(\pi, \mathcal{R})] \tag{5}$$

$$\text{s.t.} \quad \textit{Admission}(\pi, r; \theta, C) = \text{PASS}, \tag{6}$$

$$\textit{scope}(r) \subseteq \textit{scope}(C). \tag{7}$$

Here π_0 is raw order, each ladder member is dependency-valid by construction, and *Admission* jointly checks strict model-backed verification and certificate identity. The certificate is issued only after candidate verification, avoiding a circular assumption that a candidate already has a certificate. The 0.03 task-score threshold used in evaluation is a separate post-hoc quality gate, not the admission tolerance.

Exact optimization requires searching dependency-valid topological orders while evaluating a model-backed behavioral predicate. ContractKV avoids this online search through a bounded *layout ladder* (Section IV-A); empirical certification then evaluates the selected candidate under a frozen contract. We do not rely on an unproven complexity classification for this search problem.

D. Service-Template Model

In multi-tenant LLM serving, multiple tenants issue requests against a shared stable context. We abstract this as:

Definition 4 (Service Template): A **service template** $\Theta = (\mathbf{S}, \mathcal{A}, \mathcal{P})$ consists of:

- $\mathbf{S} = (s_1, \ldots, s_k)$: the ordered set of STABLE segments shared across all authorized tenants;
- $\mathcal{A} = \{a_1, \ldots, a_N\}$: the set of authorized tenant identifiers;
- $\mathcal{P}$: the frozen policy governing verification, layout selection, and certificate issuance.

Each tenant a_i appends private MUTABLE and QUERY segments to $\mathbf{S}$, forming a tenant-specific request $r_i = \mathbf{S} \| \mathbf{m}_i \| \mathbf{q}_i$. The **service locality** of a workload is the expected number of tenants sharing each template; higher locality implies greater amortization of verification cost and prefix-state residency.

The service-level objective extends Problem 1 to the multi-tenant setting: maximize aggregate SLO goodput (fraction of requests meeting a TTFT deadline Δ_{SLO}) across all tenants, subject to unauthorized-reuse isolation (private suffix state is not shared across tenants) and certificate-scope admissibility.

E. Safety and Isolation Invariants

ContractKV enforces four design invariants in the implementation and validates their failure paths through source-level instrumentation and fault injection:

(P1) Publication safety. No *candidate compiled* KV state becomes visible in the shared prefix cache unless it has passed behavioral verification *and* carries a valid certificate whose scope subsumes the current request:

$$\textit{Visible}_{\text{compiled}}(x) \implies \exists\, C : \textit{Valid}(C, x) \wedge C.\textit{status} = \text{PASS}. \tag{8}$$

(P2) Suffix isolation. The transactional commit boundary is block-aligned to the stable-prefix length and exact candidate–certificate token LCP; no MUTABLE or QUERY token is promoted:

$$\begin{aligned} L_{\text{lcp}} &= \text{LCP}(\mathbf{t}_{\text{cand}}, \mathbf{t}_{\text{cert}}), \\ L_{\text{commit}} &= B \left\lfloor \frac{\min(L_s, L_{\text{cert}}, L_{\text{lcp}})}{B} \right\rfloor, \\ N_{\text{blocks}} &= L_{\text{commit}}/B. \end{aligned} \tag{9}$$

where L_{cert} is the prompt-prefix length authorized by attestation.

(P3) Fail-closed semantics. Any verification failure, scope mismatch, timeout, or identity conflict aborts promotion with *zero publication of candidate compiled state*. The request then resumes under raw order; assuming progress of the base engine, this fallback also preserves progress. Native APC may populate raw-order cache state exactly as in unmodified vLLM.

Algorithm 1 Frozen certificate-guided layout descent.

```
Split stable text into contiguous units; freeze separators
Build L0–L3 candidates by stable-priority topological sort
Break ties by original index; retokenize and audit each prompt
for all ℓ in frozen eligible levels, high to low do
    Compile candidate π_ℓ; require P(π_ℓ) ≥ P(π_raw) + B
    Run compact teacher-forced attestation
    if attestation is PASS and scope is valid then
        PSTP commit; return π_ℓ
    end if
end for
return raw order L0 with no shared-state publication
```

(P4) Tenant isolation. For tenants $a_i \neq a_j$ under the same template Θ, private observations, request identifiers, and candidate namespaces remain disjoint. Cross-tenant KV reuse is confined to the authorized stable prefix $\mathbf{S}$; this is a cache-state isolation claim, not a claim about the surrounding application's security boundary.

IV. ContractKV Design

ContractKV resolves Problem 1 through a four-stage pipeline—*compile*, *certify*, *promote*, *amortize*—that operates entirely within the serving engine's existing prefix-cache substrate. No hardware modification, custom kernel, or external oracle is required. We present each stage in turn.

A. Cache-Aware Layout Compiler

The **Cache-Aware Layout Compiler** (CALC) transforms request r into a bounded ladder of dependency-valid candidates. The implemented candidate constructor is prefix-oriented rather than an online search over the live cache: L0 is raw order; for the full Llama calibration, stable content is split into $K = 16$ contiguous character-preserving units and L1/L2/L3 select the first $\lfloor r_\ell K \rfloor = 4/12/16$ eligible units for $r_\ell = 0.25/0.75/1.0$. A stable-priority topological sort then places selected units as early as declared SDG edges permit, breaking ties by original segment index. Frozen separators are inserted, the complete candidate is retokenized, and an exact rendered-prompt audit must pass. Qwen and Mistral evaluate a full compiled candidate, denoted L3*, behind the same per-session certificate but do not claim separately calibrated L1/L2 semantics. Resident state affects expected reuse and admission economics, not the construction order of these frozen candidates.

Prospective qualification. For a new model, every family–level pair is evaluated on a source-disjoint development split. The highest level satisfying completion, future-turn quality, scope, output-length, physical-reuse, and suffix-isolation gates becomes the frozen family maximum. The eligible descent set, policy hash, and qualification hash are frozen *before* the sealed held-out split is opened. Held-out outcomes never change the ladder or thresholds.

Runtime descent. The compiler tries only levels in the frozen eligible descent set. Each candidate must add at least one reusable block and pass strict verification and scope validation; otherwise it descends, eventually returning L0. Algorithm 1 is the implemented admission path.

The topological construction costs $O(m \log m)$ for m segments. Qualification is also lightweight in our measured onboarding: 25 development sessions over five families and four ladder cells consume 0.359 GPU-hours; the frozen manifest is 1,678 B and runtime lookup is 0.125 μs mean (0.200 μs p95).

B. Scoped Behavioral Certification

A dependency-valid layout is *necessary but not sufficient* for safe publication. Transformer attention is position-sensitive: permuting segments changes their absolute and relative positions, potentially shifting the output distribution even when all dependencies are satisfied. ContractKV therefore requires *empirical behavioral certification* before any compiled layout enters the shared cache.

In-engine teacher-forced verification. Given candidate and raw prompts, the verifier first obtains the raw reference output path $\mathbf{y} = (y_1, \ldots, y_{L_y})$. It then evaluates both prompts on the same teacher-forced history $\mathbf{y}_{<i}$. The admission contract is **strict top-1 agreement** along this generation path:

$$\begin{aligned} \forall i \le L_y: \quad & \arg\max p_\theta(\cdot \mid \mathbf{c}_{\pi^*}, \mathbf{y}_{<i}) \\ & = \arg\max p_\theta(\cdot \mid \mathbf{c}_{\pi_{\mathrm{raw}}}, \mathbf{y}_{<i}). \end{aligned} \tag{10}$$

If every position agrees and the output path completes under the frozen task budget, the verifier emits PASS; otherwise it fails closed. L_y is the verified output-path length. The distinct value L_{cert} is the stable prompt-prefix length authorized by that pass. The 0.03 task-score gate reported later is a post-hoc evaluation threshold, not a relaxed admission tolerance.

Compact attestation. The verification result is transmitted as a *compact attestation* rather than a full token-by-token trace:

$$\mathcal{A} = (\textit{decision},\ \textit{tx_id},\ L_{\mathrm{cert}},\ \textit{policy_hash},\ \textit{timestamp}). \tag{11}$$

This five-field record replaces prompt-logprob payloads that would otherwise leave the engine core. In a controlled wire measurement, the mean compact HTTP output is 387 B versus 831,262 B for the corresponding full prompt-logprob response, a **2,148× reduction**; the largest serialized compact engine-core record is 231 B. The attestation channel therefore keeps per-token logprob tensors off the scheduler egress path while retaining the decision and scope identity needed by PSTP.

Certificate scope binding. A behavioral certificate is an empirical attestation with a versioned validity key:

$$C = \langle M, T_k, V, H_s, L, P, Q, A, E \rangle, \tag{12}$$

where the fields bind model identity M, tokenizer/chat-template identity T_k, immutable template version V, stable-content hash H_s, layout hash L, verification-policy hash P, prospective-qualification hash Q, authorization domain A, and lifecycle epoch E. A request may reuse state only when all fields match and its scope is subsumed by C; any mismatch invalidates the path and returns raw order.

Scope modes. The scope type is one of OBSERVATION, QUALIFIED-FAMILY, or SERVICE-TEMPLATE. Observation scope binds exact content and epoch. Qualified-family scope is selected on a source-disjoint development split; the family level, eligible descent set, policy hash, and qualification hash

are frozen before sealed held-out evaluation. Service-template scope further binds one immutable deployment object and authorization domain. These are empirical deployment scopes, not universal claims over arbitrary future inputs.

C. Prefix-Selective Transactional Promotion

Even with a valid certificate, the act of publishing KV state into a shared prefix cache must be *transactional*: either the entire verified stable prefix becomes atomically visible, or nothing changes. ContractKV enforces this through **Prefix-Selective Transactional Promotion** (PSTP).

Block-aligned commit boundary. Given block granularity B, stable-token boundary L_s, authorized boundary L_{cert}, and the exact candidate–certificate token LCP, PSTP computes:

$$L_{\text{lcp}} = \text{LCP}(\mathbf{t}_{\text{cand}}, \mathbf{t}_{\text{cert}}),$$
$$\textit{Commit}(r) = B \left\lfloor \frac{\min(L_s, L_{\text{cert}}, L_{\text{lcp}})}{B} \right\rfloor. \tag{13}$$

Thus publication cannot cross the stable boundary, the authorized boundary, or an exact token mismatch; the floor operation excludes partial blocks.

Ephemeral candidate state. Between compilation and commit, all KV states for the candidate layout reside in an *ephemeral namespace* invisible to the prefix-cache index. Other requests cannot observe, match against, or evict this state. Only upon successful commit does the state migrate to the shared namespace and become eligible for prefix matching.

Abort paths. PSTP defines five abort conditions, each resulting in *zero publication of candidate compiled state*:

1) NON-PASS: verification returns FAIL or remains incomplete;
2) DUPLICATE: a transaction with the same *tx_id* already committed;
3) TIMEOUT: verification exceeds the frozen hard deadline Δ_{deadline};
4) IDENTITY MISMATCH: model, policy, or layout hash differs from the certificate;
5) SCOPE INVALID: the request falls outside the certificate's validity scope.

Upon abort, candidate compiled state is discarded and the request proceeds under raw-order semantics. The system never publishes partially verified or scope-invalid compiled blocks. Raw execution can subsequently populate native APC state, so the invariant is zero *candidate publication*, not zero change to the entire cache domain. Control-plane work may still add latency—the zero-locality negative control measures this overhead explicitly—so the guarantee is behavioral and transactional, not performance monotonicity.

D. Service-Template Certificate

In multi-tenant LLM serving, per-session verification scales with the number of tenants. The **service-template certificate** instead combines two layers: prospective workflow qualification and deployment attestation for one immutable template version. A service-template certificate does not independently prove equivalence for arbitrary future queries. It instantiates a prospectively qualified model–workflow–layout route for one immutable template version and amortizes its deployment attestation across authorized requests.

Template registration. When a qualified service template $\Theta = (\mathbf{S}, \mathcal{A}, \mathcal{P})$ is deployed, ContractKV attests the concrete stable content and publishes C_Θ only after PSTP succeeds. The certificate binds all nine identity, layout, policy, qualification, authorization, and epoch fields above.

Request-scope check. For each subsequent request $r_i = \mathbf{S} \| \mathbf{m}_i \| \mathbf{q}_i$ from an authorized tenant $a_i \in \mathcal{A}$, ContractKV performs a lightweight *request-scope check* rather than full re-verification:

1) Confirm that the stable prefix of r_i matches $\mathbf{S}$ (content hash equality);
2) Confirm that $a_i \in \mathcal{A}$ (authorization);
3) Confirm that model, tokenizer, template version, qualification, policy, and layout hashes match C_Θ;
4) Confirm that no lifecycle event (model update, tokenizer change, policy revision) has invalidated C_Θ.

If all checks pass, the request reuses the template's certified prefix state without per-request verification. The runtime operation is a bounded set of hash and authorization checks; Section VI-D measures its amortized cost directly rather than assuming it is free.

Lifecycle invalidation. The template certificate is invalidated—and all dependent prefix state evicted—upon any of the following events:

- Model weight update (weight hash change);
- Tokenizer revision (vocabulary or merge-rule change);
- Policy modification (strict token-path contract or scope-mode change);
- Template content update (any segment in $\mathbf{S}$ modified);
- Explicit administrative revocation.

After invalidation, the next request triggers re-registration. This lifecycle model ensures that certificates never outlive their empirical validity boundary.

Amortization analysis. For N authorized sessions or tenants sharing one template and T turns per session, session scope performs $O(N)$ full attestations plus $O(NT)$ certificate lookups. Template scope performs $O(1)$ full attestation plus the same $O(NT)$ bounded lookups. At 25-tenant locality, this change of certificate object lowers measured unhidden verification cost from 25.05 to 3.24 ms/request (7.7×) without materially changing new-prefill work (Section VI-D).

V. IMPLEMENTATION

We implement ContractKV as a middleware layer atop **vLLM 0.10.2** [1], targeting the V1 scheduler path. The implementation comprises three components: a layout compiler integrated at the request-admission boundary, a verification executor that shares the model backend, and a transactional promoter hooked into the prefix-cache block allocator.

Scheduler integration. The layout compiler intercepts each incoming request before scheduler admission. It constructs the SDG from application-declared segment boundaries, stability types, and dependency annotations, executes CALC, fully

retokenizes the rendered candidate, and emits the compiled token sequence. ContractKV does not modify attention kernels, model weights, or sampling logic. It adds scheduler-side admission hooks and cache-index publication logic to the vLLM V1 block-management path.

Physical prefix-hit and new-prefill token counts are recorded at the scheduler's block-allocation boundary. For each request, the scheduler reports the number of blocks served from the resident prefix cache versus blocks requiring fresh prefill computation. These counters provide ground-truth physical reuse metrics independent of timing inference.

Verification executor. The behavioral verifier reuses the serving model backend and weight instance. Session-scoped verification may execute during a soft hiding window Δ_{slack}; its unhidden charge is

$$C_{\text{unhidden}} = \max(C_{\text{verify}} - \Delta_{\text{slack}}, 0). \tag{14}$$

Crossing this soft window does not abort verification. A separate frozen hard timeout enforces $C_{\text{verify}} > \Delta_{\text{deadline}} \Rightarrow$ ABORT. The primary Qwen3-8B, Mistral, and Qwen3-14B protocols freeze 2,000 ms slack, while the broad Llama qualification freezes 6,000 ms; Section VII-B sweeps 0–3,000 ms to expose this boundary.

The compact attestation is serialized as a fixed-size record and stored in an in-memory certificate table indexed by the nine-field versioned key. The source-level egress path emits no prompt-logprob tensor; request and transaction identity remain scheduler-local. Certificate lookup uses a hash map.

Transactional promoter. PSTP is implemented as a commit gate in the prefix-cache block-management path. Candidate KV blocks remain excluded from shared prefix matching until a valid PASS. The scheduler thread then calls the cache manager's registration path for exactly $N_{\text{blocks}} = L_{\text{commit}}/B$ certified blocks. This insertion of certified block hashes into the shared prefix index is the linearization point; the KV tensors are already resident and are not copied during commit. On abort, candidate registration is skipped and native cleanup proceeds.

Fresh-process experimental framework. To eliminate cross-cell cache contamination, our evaluation harness creates *independent server processes* for each experimental cell. Each cell receives a dedicated GPU assignment, port, compile cache, and metrics log. The request runner alternates policy order (raw-order vs. ContractKV) and GPU assignment across repetitions to control for thermal and scheduling bias. Workload traces are SHA-frozen before measurement; partial cell resume is forbidden. This framework ensures that reported metrics reflect steady-state serving behavior rather than warm-start artifacts.

VI. EVALUATION

We evaluate ContractKV across four model configurations from three families, five task workflows, and multi-tenant service deployments. Primary task-quality and common-scope experiments require 100% completion, zero policy-attributable quality violations, zero scope-invalid admissions, and zero unauthorized cross-tenant KV reuse. Open-loop service cells instead report measured completion and drop rates explicitly.

TABLE II
MEASURED CONTROL-PLANE LATENCY. COMPILER ROWS CONTAIN 36,000 INVOCATIONS; SCOPE CHECKS ARE LIVE SERVICE-CELL MEANS; PSTP AND INVALIDATION ROWS ARE SOURCE-LEVEL EVENTS.

Operation	Mean (ms)	p95 (ms)
CALC compilation	0.389	0.819
Residency-ledger lookup	0.249	0.507
Template scope check	0.109	0.111
PSTP finalization	0.636	0.685
Lifecycle invalidation	0.124	0.162

TABLE III
GLOBAL ALL-WORKFLOW RESULTS (3 FRESH-PROCESS REPETITIONS; RAW FALLBACK AND UNHIDDEN VERIFICATION INCLUDED). GOODPUT USES EACH PROTOCOL'S FROZEN SLO RATHER THAN A POST-HOC COMMON THRESHOLD.

Metric	Qwen 8B	Qwen 14B	Mistral	Llama
Net mean TTFT red.	37.81%	44.69%	31.72%	29.41%
New-prefill red.	59.13%	60.57%	39.46%	30.97%
Net SLO goodput (×)	2.64	2.92	1.24	1.19
Cert. coverage	68%	68%	50%	50%
Quality violations	0	0	0	0
Scope-invalid adm.	0	0	0	0
Completion	100%	100%	100%	100%

A. Experimental Setup

Hardware and software. One server with two NVIDIA RTX 3090 GPUs (24 GB each), 128 GB system RAM, running vLLM 0.10.2 with automatic prefix caching enabled. Qwen3-8B, Qwen3-14B, and Mistral use an 8,192-token limit; the broad Llama protocol uses 16,384 tokens. Qwen3-14B uses tensor parallelism 2; the other models use one GPU. Primary paired serving uses concurrency 2.

Models. Four configurations spanning three model families and the 7–14B parameter class: **Qwen3-8B** (primary), **Qwen3-14B** (TP=2 scale-up), **Mistral-7B-Instruct-v0.3**, and **Llama-3.1-8B-Instruct** (adaptive qualified policy). Each model uses a prospectively frozen layout policy with no cross-model threshold transfer.

Workload. Five service families (RULER [19], HotpotQA, NarrativeQA, Qasper, RepoBench) span long-context retrieval, reasoning, and code [26]. Each model evaluates 25–50 sealed sessions *in total*, balanced across the five families, with 10 turns per session and three fresh-process repetitions.

Protocol. Policy order and GPU assignment are rotated across repetitions. We report *net* mean TTFT (observed TTFT plus C_{unhidden}), new-prefill tokens, SLO goodput, and certificate coverage. Task-specific output budgets are frozen from development profiling; the 64-token deterministic marker used in service-load cells isolates queueing, prefix reuse, and TTFT and is not a production output cap. Full task metrics are measured in separate paired quality runs. Session-cluster bootstrap confidence intervals resample fresh-process repeats and paired sessions.

B. Global All-Workflow Results

Table III and Fig. 2 report the primary all-workflow results across four models. ContractKV delivers consistent latency reductions with zero safety violations in every cell.

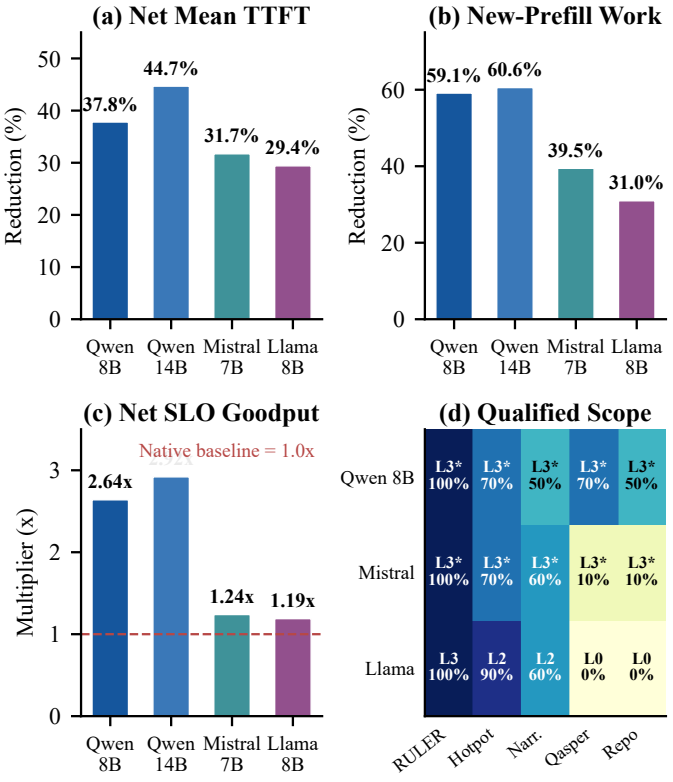


Fig. 2. Global results and qualification map. (a–c) ContractKV reduces net mean TTFT by 29.41–44.69% and new-prefill work by 30.97–60.57%; SLO multipliers are net-amortized at the frozen protocol deadline. (d) Compiled coverage is model–workflow dependent; L0 and runtime fallback remain in every global row. L3* denotes a full compiled candidate without separately calibrated L1/L2.

Three observations stand out. *First*, prospective qualification yields positive global gains for every configuration across three model families and two parameter scales. *Second*, deterministic new-prefill reduction exceeds net TTFT reduction (e.g., 59.13% vs. 37.81% for Qwen3-8B), exposing the verification and serving costs already charged by the net metric. In a same-runtime decomposition, Qwen3-8B measures 1,396.4 ms under raw APC, 623.0 ms with the frozen layout alone, and 868.4 ms with full online ContractKV; Mistral measures 1,102.5, 695.8, and 752.8 ms. *Third*, Qwen3-14B reaches 44.69% net TTFT and 60.57% prefill reduction under TP=2, a limited scale-up point rather than a cluster claim.

Coverage and fallback accounting. The global net metric includes admitted compiled requests, runtime verifier fallback, families prospectively frozen at L0, and verification time not hidden by the declared slack. Thus 50% coverage on Mistral and Llama does not mean that half of all requests perform an expensive failed verification: many are routed directly to L0 by the frozen family manifest. Figure 2(d) makes this composition explicit. The only materially negative compiled family cell is Mistral RepoBench at 10% sparse coverage (-2.22% net TTFT); L0 cells remain within approximately $\pm 1\%$ raw-run noise. This motivates an explicit locality/economics operating region rather than post-hoc deletion from the global row.

TABLE IV
COMMON FROZEN RULER SCOPE: 30 IDENTICAL SESSIONS, 10 TURNS, 3 FRESH-PROCESS PAIRS PER MODEL. ALL MODELS CONSUME THE SAME SEALED TRACE (SHA-256 COMMITTED).

Metric	Qwen 8B	Mistral	Llama
Net mean TTFT red.	86.64%	87.41%	85.97%
95% CI	[86.16, 87.10]	[87.06, 87.79]	[85.63, 86.29]
New-prefill red.	96.71%	95.86%	95.45%
SLO req/s (raw→CKV)	2.42→6.33	2.67→8.31	3.42→14.42
Goodput multiplier	2.61×	3.11×	4.22×
Cert. coverage	100%	100%	100%
Quality violations	0	0	0
Paired outputs	810	810	810

C. Cross-Model Convergence on Common Scope

A critical question is whether ContractKV's gains are model-specific or converge under identical conditions. We answer this with a *common frozen RULER scope*: 30 source-group-disjoint sessions replayed on three models under the same L3 layout, with no requalification.

Table IV shows close convergence: the model-stratified macro TTFT reduction is **86.67%** with a 95% CI of [86.44, 86.90], and the three estimates lie within a 1.44-percentage-point band. Because the models consume the same frozen sessions and layout, this result controls sample asymmetry and supports the conclusion that reusable-prefix structure dominates within a prospectively qualified common scope.

The SLO-goodput multiplier varies (2.61–4.22×) because each model has a different raw decode rate; this reflects baseline throughput differences, not mechanism strength. As a portability check, the same offline-frozen safe layouts reduce real SGLang 0.4.9 RadixAttention TTFT by 54.93% on Qwen and 35.53% on Mistral. This B4 cell shows that the compiler output is consumable by another prefix substrate; it is not full online ContractKV because SGLang does not run PSTP or issue certificates.

D. Service-Template Certificate

The service-template certificate is ContractKV's most distinctive design contribution. We validate it along two axes: quality preservation under fan-out, and verification amortization.

Quality preservation (P0-A). Five deployed templates (RULER, HotpotQA, NarrativeQA, Qasper, RepoBench) are each consumed by 10 authorized tenants for 10 turns under three paired fresh processes. This yields 1,350 paired future-turn executions ($5 \times 10 \times 9 \times 3$), clustered by template, tenant, and process rather than 1,350 independent examples. Using the frozen task metrics and original task-specific output budgets, Fig. 4(a) reports a raw-minus-ContractKV quality drop of **exactly zero** with CI [0, 0], zero policy violations above 0.03, zero unauthorized reuse, and zero scope-invalid admissions. Qasper correctly follows fail-closed raw fallback; the remaining four families receive shared template certificates with no measurable quality consequence.

Amortization ablation (P0-B). Fig. 4(b) isolates the economic effect of certificate-object granularity. At identical 25-tenant locality and 80% load, switching from session-scoped to template-scoped certificates reduces verifier calls

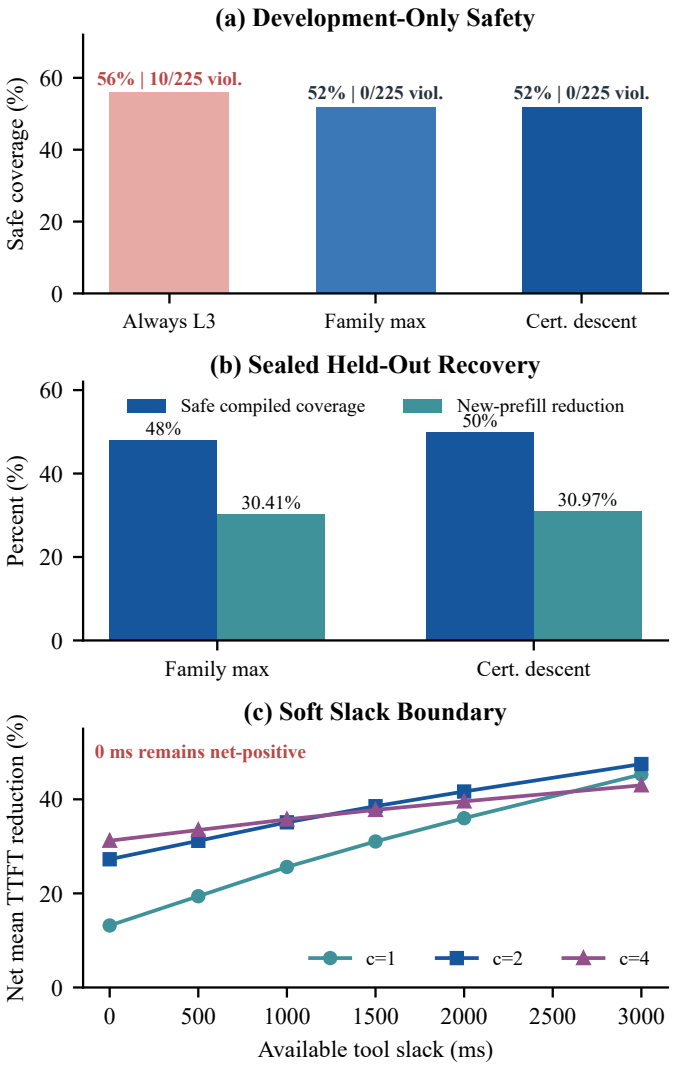


Fig. 3. Safety and slack sensitivity. (a) Development-only ladder calibration: Always-L3 violates 10/225 turns, whereas family qualification and certificate descent have zero violations. (b) On sealed held-out sessions, descent raises safe compiled coverage from 48% to 50% and new-prefill reduction from 30.41% to 30.97%. (c) Net TTFT charges measured verification beyond the soft slack window and remains positive at 0 ms for the evaluated session-scoped trace.

from 125 to 5 (25×), raises fan-out from 9 to 225 requests per certificate, and reduces unhidden verification cost from 25.05 to 3.24 ms/request (7.7×). Critically, the serving work is unchanged: new-prefill tokens differ by only 8.5 (0.37% of prompt), and mean TTFT, p99, and SLO goodput are identical. *Template-scoped certificates do not create new reuse opportunities; they amortize the same safe decision over the correct deployment object.*

VII. Service-Level Evaluation

We now evaluate ContractKV under multi-tenant service workloads that stress SLO goodput, consolidation density, working-set pressure, and unauthorized-reuse isolation. Arrival timestamps are generated once and replayed identically across D0/D1/D2. The unit-rate trace is exponentially distributed; 100% load is the frozen raw-APC calibration rate, and each cell contains 1,125 future requests over three fresh processes.

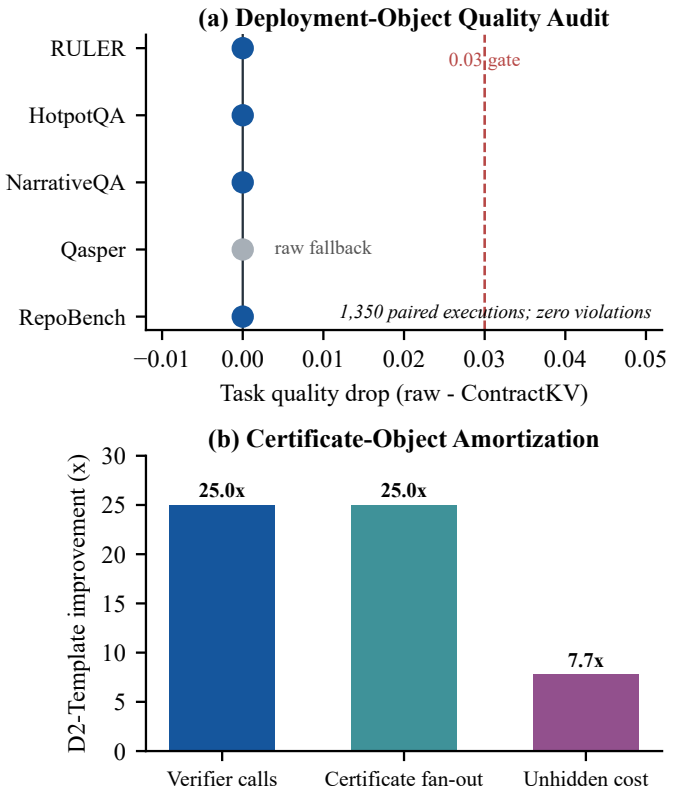


Fig. 4. Service-template certificate validation. (a) Raw-minus-ContractKV task-score drop across 1,350 paired future-turn executions is zero for all five families (Qasper uses raw fallback); the dashed line is the 0.03 violation gate. (b) Template-scope amortization reduces verifier calls 25× and unhidden cost 7.7× without changing serving work.

The load sweep uses Poisson open-loop arrival; the 80% consolidation point adds the frozen ON/OFF burst overlay. Across the service study we execute 288,495 marker-bearing future requests, while task behavior is assessed separately on 2,700 task-bearing raw/ContractKV requests.

A. Service-Level Performance

Fig. 5 reports three complementary views of service-level gain.

Load scaling (Fig. 5a). Under 25-tenant service-template sharing at 80% load and a 500-ms deadline, Qwen3-8B mean TTFT falls by 92.62% (session-cluster 95% CI: 91.68–93.31%). Absolute SLO goodput rises from 0.173 to 0.831 requests/s, a gain of 0.658 requests/s (95% CI: 0.539–0.765) and a 4.81× point multiplier. At 120% load, raw and ContractKV goodput are 0.153 and 1.246 requests/s. A matched Mistral 25-tenant cell reduces mean TTFT by 54.35%, p99 by 29.65%, and new-prefill by 50.75%, with a 0.340 requests/s absolute 500-ms goodput gain (clustered CI: 0.129–0.538). We report absolute curves rather than ratios from near-zero denominators.

Consolidation density (Fig. 5b). Scaling from 10 to 40 tenants under 100% authorized sharing, ContractKV sustains ∼11 SLO-compliant req/s; at 40 tenants, goodput rises from 1.064 to 10.984 requests/s while mean TTFT falls from 1,364.5 to 90.8 ms. Measured Jain fairness [22] rises from 0.666 to

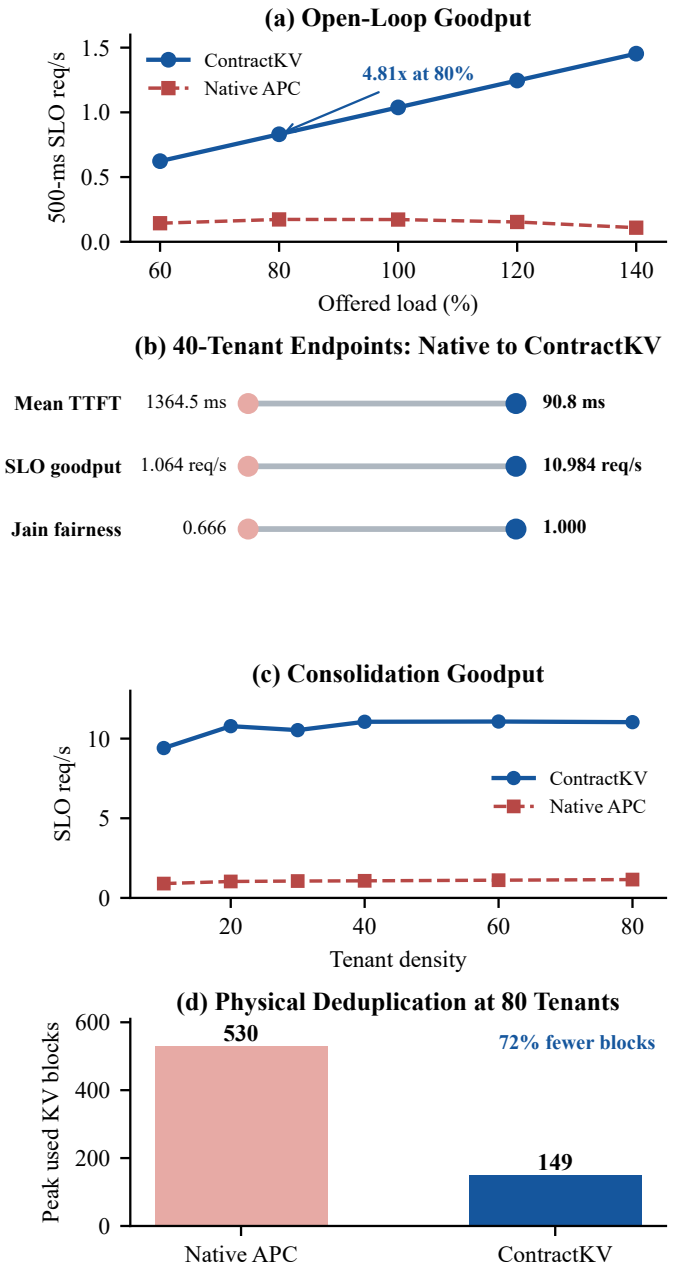


Fig. 5. Service-level evaluation. (a) Absolute 500-ms SLO goodput across offered load; at 80%, raw order and ContractKV achieve 0.173 and 0.831 requests/s. (b) Direct 40-tenant endpoints for mean TTFT, goodput, and Jain fairness. (c) SLO goodput across 10–80 tenants. (d) At 80 tenants, peak used KV blocks fall from 530 to 149 (72%).

1.000 under the same trace; we report this association without attributing it to a separate fairness mechanism.

Density scaling (Fig. 5c). At 80 tenants, peak used KV blocks fall from 530 to 149 (72%) and free blocks rise from 1,856 to 2,237; SLO-compliant throughput is 1.152 requests/s for raw order and 11.033 requests/s for ContractKV. These are allocator-observed physical-block counts rather than a conversion inferred from new-prefill tokens.

B. Operating Region and Boundaries

Fig. 6 characterizes the *operating region*—the conditions under which ContractKV's gains materialize.

Tool-slack sensitivity (Fig. 3b). Using measured canary latency and live TTFT, we recompute net latency at 0, 500, 1,000, 1,500, 2,000, and 3,000 ms available slack. At zero slack, session-scoped ContractKV remains net-positive by 13.18%, 27.26%, and 31.20% at concurrency 1, 2, and 4; at 2,000 ms the reductions are 35.99%, 41.66%, and 39.56%. Thus slack expands the benefit but is not assumed free. Low-locality or short-prefix cells can still be negative, while service-template scope reduces unhidden verification to 3.24 ms/request through deployment-object fan-out.

Observed sharing-ratio knee (Fig. 6a). We sweep the fraction of requests authorized under a shared template over 0, 25, 50, 75, and 100%. The measured knee lies between 50% and 75% sharing: once enough requests consume the certified deployment object, its prefix remains resident and TTFT reduction exceeds 85%. This is an operating-region observation for the evaluated trace and cache size, not a universal 75% threshold.

Working-set residency (Fig. 6b). Under Zipf arrival (modeling realistic skewed access), ContractKV retains 73–94% TTFT reduction even at 50 active templates because popular templates remain cache-resident. Under round-robin arrival, gains collapse to zero when reuse distance exceeds cache capacity (25+ templates). The asymmetry confirms that ContractKV's benefit is proportional to the ratio of cache capacity to effective working-set size under the actual access distribution.

Negative control (Fig. 6c). Under zero-locality workloads (unique stable content per session), ContractKV imposes 1.6–8.7% TTFT overhead from control-plane processing with no compensating reuse benefit. The overhead grows with load. This negative control defines a deployment operating region: an implementation should use measured locality and verification economics to select ContractKV, while retaining raw order outside the profitable region. Our prototype exposes these measurements but does not claim a perfect online benefit predictor.

C. Safety, Lifecycle, and Economics

Table V consolidates the safety, lifecycle, and verification-economics evidence across all service-level experiments.

Safety: zero unauthorized reuse, zero dynamic-suffix leakage, and zero scope-invalid admissions across all 34 correctness rows. *Lifecycle*: all 10 simulated invalidation events (model update, tokenizer revision, policy change, template content update, administrative revocation, and combinations) invalidate dependent certificates and trigger re-registration, with zero stale-certificate admissions. *Economics*: 71 of 84 measured live lanes have positive amortized economics, while the frozen certificate policy accepts 82 lanes and rejects two to raw order. We therefore use these data as an operating-region analysis, not as evidence of a perfect online benefit predictor. *Quality*: across 1,350 paired future outputs from five deployed service templates, the frozen task-score aggregate is 0.356 for both raw and D2-template serving, with zero policy-attributable violations; Qasper remains on raw fallback.

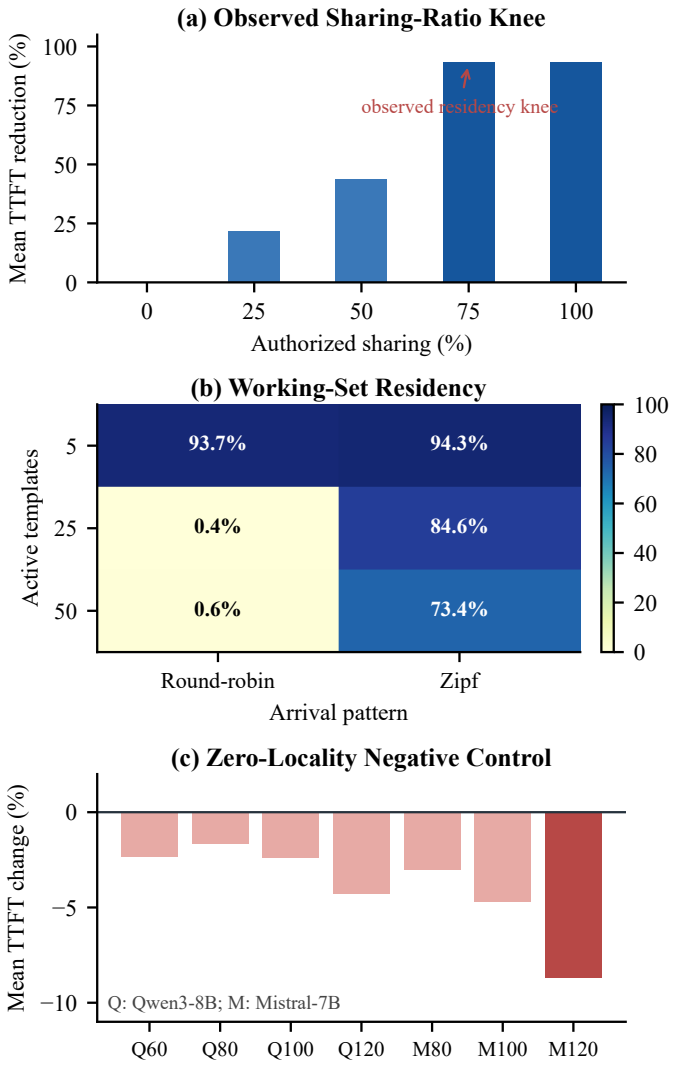


Fig. 6. Operating region and boundaries. (a) The measured locality knee lies between 50% and 75% sharing. (b) Working-set residency under Zipf versus round-robin arrival. (c) Negative control: 1.6–8.7% overhead under zero locality.

VIII. Related Work

Prefix caching substrates and serving architectures. PagedAttention [1] introduced non-contiguous KV block allocation, and RadixAttention [2] indexes reusable prefixes in SGLang. Prompt Cache [7] exposes modular attention reuse. CacheBlend [8] and concurrent SparseX [9] recover non-prefix reuse through selective recomputation, whereas ContractKV changes a dependency-valid prompt layout and admits it through scoped behavioral evidence before publication. DistServe [4], Splitwise [27], Mooncake [16], AlpaServe [5], TetriInfer [24], and FlexGen [25] optimize scheduling, disaggregation, or offloading around these cache substrates.

The capability boundary is therefore precise: native APC/RadixAttention index physical prefixes; Prompt Cache exposes schema-defined modules; CacheBlend/SparseX reuse non-prefix state through recomputation; ContractKV alone combines an SDG-constrained layout, scoped model-backed admission, and fail-closed publication of candidate compiled state.

TABLE V
Safety, lifecycle, and verification-economics summary across all service-level experiments (Qwen3-8B, 25–80 tenants, Poisson open-loop arrival).

Dimension	Result	Evidence
Unauthorized cross-tenant reuse	0 events	34 aggregate correctness rows
Dynamic-suffix reuse	0 events	All load, arrival, density configs
Scope-invalid admissions	0 events	Full lifecycle gate
Lifecycle events tested	10	Model, tokenizer, policy, content, revoke
Stale-certificate admissions	0	Post-invalidation re-registration
Measured positive-economics lanes	71/84	Operating-region characterization
Certificate outcomes	82 pass / 2 fallback	Frozen policy; no post-hoc thresholding
Verifier-call reduction	25×	Template vs. session scope
Unhidden cost reduction	7.7×	25.05→3.24 ms/req
Quality preservation	0 delta	1,350 paired outputs, 5 families

KV compression and eviction. H2O [11] and Scissorhands [12] reduce KV memory footprint by evicting low-importance tokens. StreamingLLM [13] identifies attention-sink tokens to enable infinite-length streaming, while SnapKV [14] and InfiniGen [15] use observation-based selection to retain only task-critical KV entries. CacheGen [10] compresses KV states for network transfer. These techniques optimize *within* a fixed prompt order—they reduce the cost of computing a prefix but do not create new reuse opportunities from structural reordering. ContractKV's layout compilation is complementary: it increases the *length* of the reusable prefix, while compression reduces the *cost per token* of that prefix. The two approaches compose naturally.

Prompt optimization. Prompt Cache [7] exposes reusable prompt modules, while application-level prompt construction can place stable modules earlier to improve reuse. CacheGen [10] instead compresses KV state for transfer. ContractKV's distinct contribution is a control plane: it checks declared dependencies, obtains model-backed evidence within an explicit scope, and publishes compiled state at a transactional linearization point. Failed verification produces zero publication of candidate compiled state.

Service computing and QoS management. QoS-aware multi-tenant service selection [23] addresses differentiated service requirements at the orchestration layer. ContractKV contributes a lower-level cache-lifecycle abstraction: a service-template certificate is an explicitly scoped, versioned reuse contract that joins verification economics with SLO-driven serving. Its fail-closed publication and unauthorized-reuse isolation are evaluated within one cache domain; distributed transactions across KV pools remain future work.

IX. Discussion and Conclusion

Why and when ContractKV helps. The strongest operating point combines three conditions: a large stable segment set, repeated authorized consumption, and enough cache residency for the compiled prefix to survive. The qualification map in Fig. 2(d) shows that this is a *model–workflow property*, not a universal promise: RULER admits compiled layouts across

all models, retrieval workloads admit model-dependent levels, and order-sensitive families may freeze at L0. An always-L3 policy exposes more candidate token work but causes 10/225 development-turn violations; prospective family qualification and runtime descent remove those violations while retaining useful coverage. ContractKV therefore turns layout aggressiveness into a measured control decision: profitable certified scopes obtain physical reuse, while weak-locality, order-sensitive, or unverifiable scopes preserve raw behavior. The observed association between token-path robustness and safe layout level is useful for qualification, but we do not claim it as an architectural cause.

Limitations. ContractKV targets repeated, authorized stable templates with enough locality and residency to amortize certification; zero-locality services should leave it disabled. Its certificates are empirical attestations, not formal semantic proofs, and service-template scope instantiates only a prospectively qualified route for one immutable version. The current PSTP implementation provides atomic visibility within one vLLM cache domain. Extending the same semantics to a disaggregated KV pool would require distributed publication, versioned invalidation, and recovery from partial node failures; network and cross-node consistency costs are outside this evaluation. The SDG/compiler and certificate metadata are portable, but the PSTP visibility backend must be reimplemented. Our two-GPU, 80-tenant study therefore establishes single-domain service behavior rather than production cluster scalability.

Conclusion. ContractKV shows that structured LLM services contain a large reuse opportunity that linear prefix caches alone cannot expose. Dependency-constrained compilation, scoped behavioral certification, and transactional promotion convert stable service context into physical prefix reuse: the strongest measured service point reduces mean TTFT by 92.62%, improves 500-ms SLO goodput by 4.81×, and at 80 tenants reduces peak used KV blocks by 72%. The service-template certificate further separates reuse from verification granularity, lowering unhidden verification cost 7.7× without materially changing serving work. Across all admitted scopes, quality and isolation gates remain satisfied. Fail-closed operation guarantees that failed or out-of-scope candidates publish no shared state and revert to raw-order behavior; the measured negative control makes clear that this safety guarantee does not imply zero control-plane latency overhead.

REFERENCES

[1] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Oper. Syst. Principles (SOSP)*, 2023, pp. 611–626.

[2] L. Zheng *et al.*, "SGLang: Efficient execution of structured language model programs," in *Proc. 38th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2024, doi: 10.52202/079017-2000.

[3] G. Yu, J. S. Jeong, G. Kim, S. Kim, and B. Chun, "Orca: A distributed serving system for transformer-based generative models," in *Proc. 16th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2022, pp. 521–538.

[4] Y. Zhong *et al.*, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 193–210.

[5] Z. Li *et al.*, "AlpaServe: Statistical multiplexing with model parallelism for deep learning serving," in *Proc. 17th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2023, pp. 663–679.

[6] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Gulavani, "Sarathi-Serve: Taming the serving bubble with chunked-prefills," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 211–228.

[7] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt Cache: Modular attention reuse for low-latency inference," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024.

[8] J. Yao *et al.*, "CacheBlend: Fast large language model serving for RAG with cached knowledge fusion," in *Proc. 20th Eur. Conf. Comput. Syst. (EuroSys)*, 2025, doi: 10.1145/3689031.3696098.

[9] Q. Zhang *et al.*, "SparseX: Efficient segment-level KV cache sharing for interleaved LLM serving," *arXiv preprint arXiv:2606.01751*, 2026.

[10] Y. Liu *et al.*, "CacheGen: KV cache compression and streaming for fast large language model serving," in *Proc. ACM SIGCOMM*, 2024, pp. 138–156.

[11] Z. Zhang *et al.*, "H2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Proc. 37th Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[12] Z. Liu *et al.*, "Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time," in *Proc. NeurIPS*, 2023.

[13] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *Proc. 12th Int. Conf. Learn. Represent. (ICLR)*, 2024.

[14] Y. Li *et al.*, "SnapKV: LLM knows what you are looking for before generation," in *Proc. NeurIPS*, 2024.

[15] C. Tang *et al.*, "InfiniGen: Efficient generative inference of large language models with dynamic KV cache management," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 155–172.

[16] R. Qin *et al.*, "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," *arXiv preprint arXiv:2407.00079*, 2024.

[17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learn. Represent. (ICLR)*, 2023.

[18] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.

[19] C. Hsieh *et al.*, "RULER: What's the real context size of your long-context language models?" *arXiv preprint arXiv:2404.06654*, 2024.

[20] A. Vaswani *et al.*, "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.

[21] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. NeurIPS*, 2022.

[22] R. Jain, D. Chiu, and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 26, no. 2, pp. 34–35, 1998.

[23] Y. Wang, Q. He, X. Zhang, D. Ye, and Y. Yang, "Efficient QoS-aware service recommendation for multi-tenant service-based systems in cloud," *IEEE Trans. Services Comput.*, vol. 13, no. 6, pp. 1045–1058, 2020, doi: 10.1109/TSC.2017.2761346.

[24] C. Hu, H. Huang, L. Xu, X. Chen, J. Xu, S. Chen, H. Feng, C. Wang, S. Wang, Y. Bao, N. Sun, and Y. Shan, "Inference without interference: Disaggregate LLM inference for mixed downstream workloads," *arXiv preprint arXiv:2401.11181*, 2024.

[25] Y. Sheng *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. 40th Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 31094–31116.

[26] Y. Bai *et al.*, "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2024, pp. 3119–3137.

[27] P. Patel *et al.*, "Splitwise: Efficient generative LLM inference using phase splitting," in *Proc. 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, 2024, pp. 118–132.